Advanced Programming

Lab 11, dynamic memory in classes

廖琪梅，王大兴，于仕琪，王薇

Exercise:

1. Could the program be compiled successfully? Why? Modify the program until it passes the compilation. Then run the program. What will happen? Explain the result.

```
#include <iostream>
#include <memory>
using namespace std;
int main()
{
    double *p_reg = new double(5);
    shared_ptr<double> pd;
    pd = p_reg;
    pd = shared_ptr<double>(p_reg);
    cout << "*pd = " << *pd << endl;
    shared_ptr<double> pshared = p_reg;
    shared_ptr<double> pshared(p_reg);
    cout << "*pshred = " << *pshared << endl;
    string str("Hello World!");
    shared_ptr<string> pstr(&str);
    cout << "*pstr = " << *pstr << endl;
    return 0;
}
```

Exercise:

2. Create a class Matrix to describe a matrix. The element type is float. One member of the class is a pointer(or a smart pointer) who points to the matrix data.

The two matrices can share the same data through a copy constructor or a copy assignment.

The following code can run smoothly without memory problems.

```
class Matrix{...};
Matrix a(3,4);
Matrix b(3,4);
Matrix c = a + b;
Matrix d = a;
d = b;
```

```
a is:
0 0 0 0
0 0 3 0
0 0 0 0
b is:
0 0 0 0
0 0 0 0
0 0 0 4
c is:
0 0 0 0
0 0 3 0
0 0 0 4
Before assignment,d is:
0 0 0 0
0 0 3 0
0 0 0 0
After assignment,d is:
0 0 0 0
0 0 0 0
0 0 0 4
```